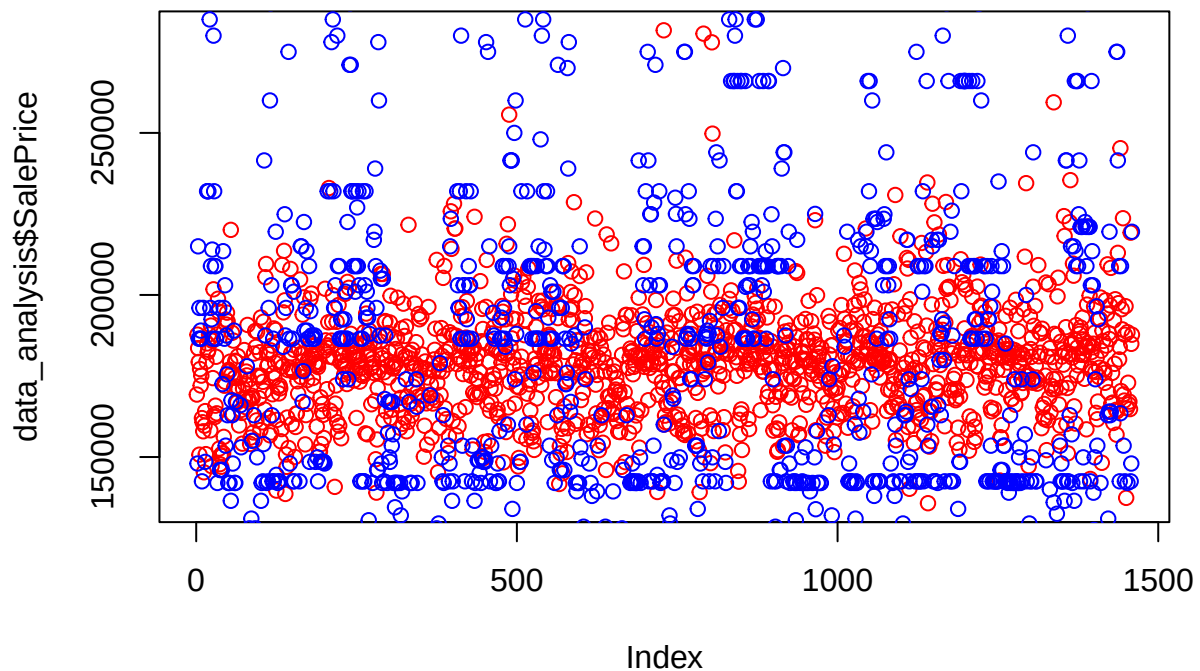


Avances 2.3

Andre Jo 22199

2025-03-14



Observamos que en la parte inferior se observa un patrón de overfitting pero en la parte de arriba no tanto. Además de eso, el modelo acertó pocas veces con la concentración de datos que existe en el rango de SalesPrice de 150,000 a 200,000.

```
EMA <- (mean(data_analysis$SalePrice-predBayes, na.rm = T)^2)
EMA
```

```
## [1] 141211223
```

```
RMSE <- sqrt(mean((data_analysis$SalePrice - predBayes)^2, na.rm = TRUE))
RMSE
```

```
## [1] 70237.08
```

A comparación del RMSE del modelo Naive Bayes, observamos que pertenece al rango de los modelos presentados anteriormente. (entrega pasada)

```

modeloRF1<-randomForest(SalePrice~.,data=train)
prediccionRF1<-predict(modeloRF1,newdata = datos)
rmseModelo10test<-RMSE(prediccionRF1,mergeprice)
mseModelo10test<-mean((prediccionRF1 - mergeprice)^2)

prediccionRF1Train<-predict(modeloRF1,newdata = datos)
rmseModelo10train<-RMSE(prediccionRF1Train,train$SalePrice)

## Warning in pred - obs: longitud de objeto mayor no es múltiplo de la longitud
## de uno menor
mseModelo10train<-mean((prediccionRF1Train - train$SalePrice)^2)

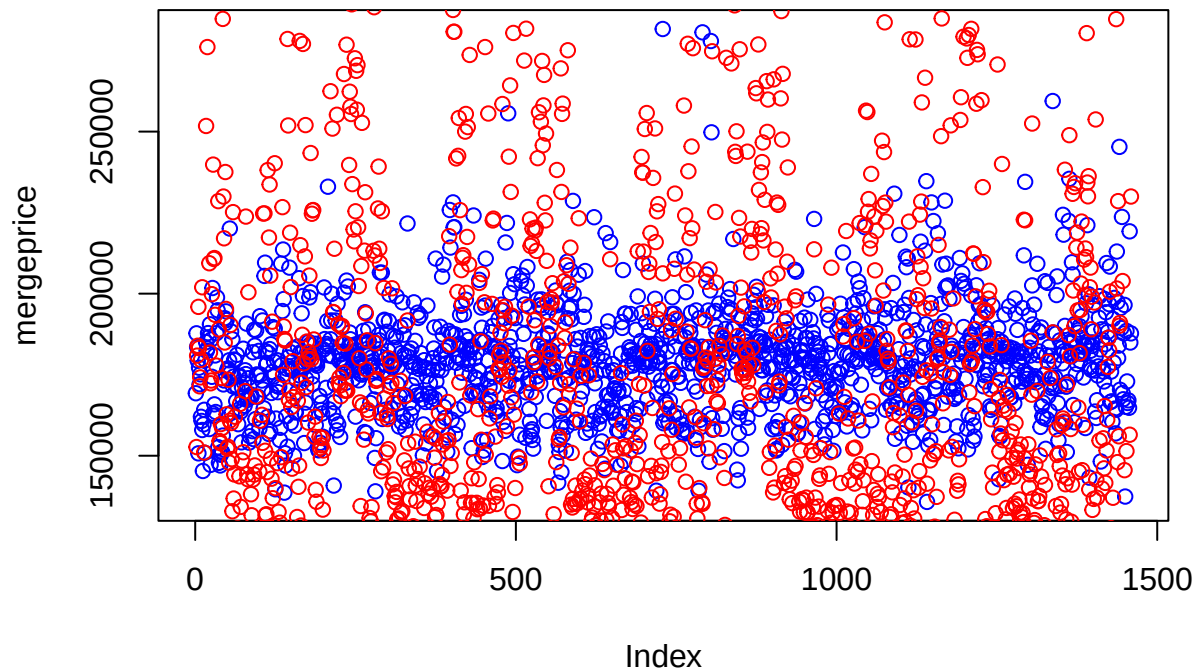
## Warning in prediccionRF1Train - train$SalePrice: longitud de objeto mayor no es
## múltiplo de la longitud de uno menor
mseModelo10test

## [1] 4711955718
rmseModelo10test

## [1] 68643.69
plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(prediccionRF1, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)

```

Predicciones vs valores originales



De las entregas anteriores, se puede observar que la gráfica que más relación tiene es el modelo de Random

Forest ya que no es un modelo que este presente el overfitting de los datos.

RMSE DE LOS SIGUIENTES MODELOS

```
CompModelos<-c( 76444.36,76444.36,76444.36,68855.19,69164.43)
names(CompModelos)<-c("Tuneado Modelo1","Tuneado Modelo2","Tuneado Modelo3", "Tuneado Modelo4","Modelo Random Forest")
```

CompModelos

##	Tuneado Modelo1	Tuneado Modelo2	Tuneado Modelo3
##	76444.36	76444.36	76444.36
##	Tuneado Modelo4	Modelo Random Forest	
##	68855.19	69164.43	

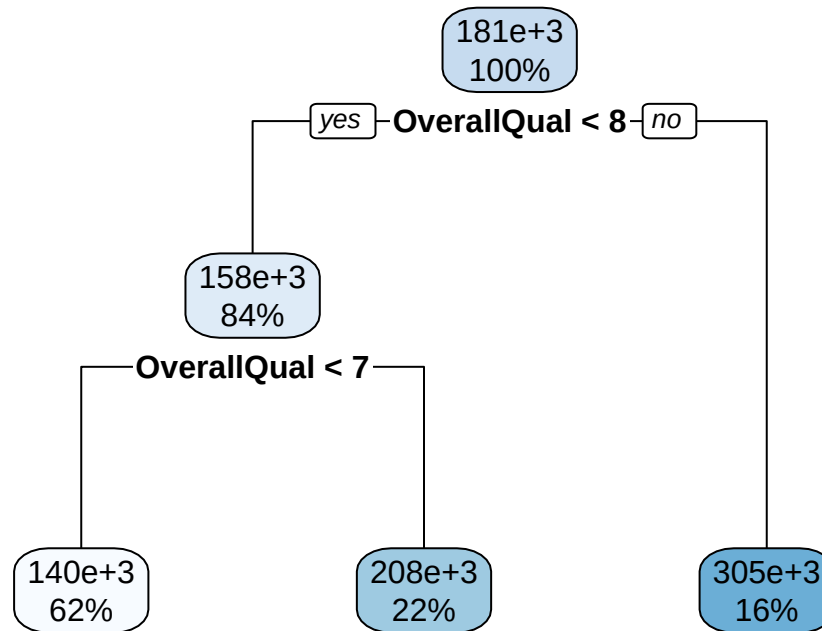
De acuerdo con la siguiente tabla a comparar y el rmse del modelo de prueba con Naive Bayes, obtenemos un RMSE de 70237.08 lo cual esta cerca ante los modelos de Random Forest y el Modelo 4. Además no observamos que la gráfica no tenga overfitting lo cual se relaciona con el modelo de Random Forest, pero observamos que Naive Bayes no acierta con los puntos que estan concentrados en el rango de 150,000 a 200,000 de la variable salePrice. Esto significa que Random Forest sigue siendo el modelo con más certeza a comparación de los otros modelos.

8.Haga un modelo usando validación cruzada, compare los resultados de este con los del modelo anterior. ¿Cuál funcionó mejor?

```
controlcv <- trainControl(method = "cv",number = 10)
modelo5<- train(SalePrice~.,data=train,
  method="rpart",
  trControl=controlcv,
  na.action=na.pass
)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
rpart.plot(modelo5$finalModel)
```



```

predModelo5 <- predict(modelo5, newdata = datos)
rmseModelo5test<-RMSE(predModelo5,mergeprice)
mseModelo5test<-mean((predModelo5 - mergeprice)^2)

predModelo5Train <- predict(modelo5, newdata = train)
rmseModelo5train<-RMSE(predModelo5Train,train$SalePrice)
mseModelo5train<-mean((predModelo5Train - train$SalePrice)^2)

```

```
mseModelo5test
```

```
## [1] 3998029436
```

```
## [1] 3998029436
```

```
rmseModelo5test
```

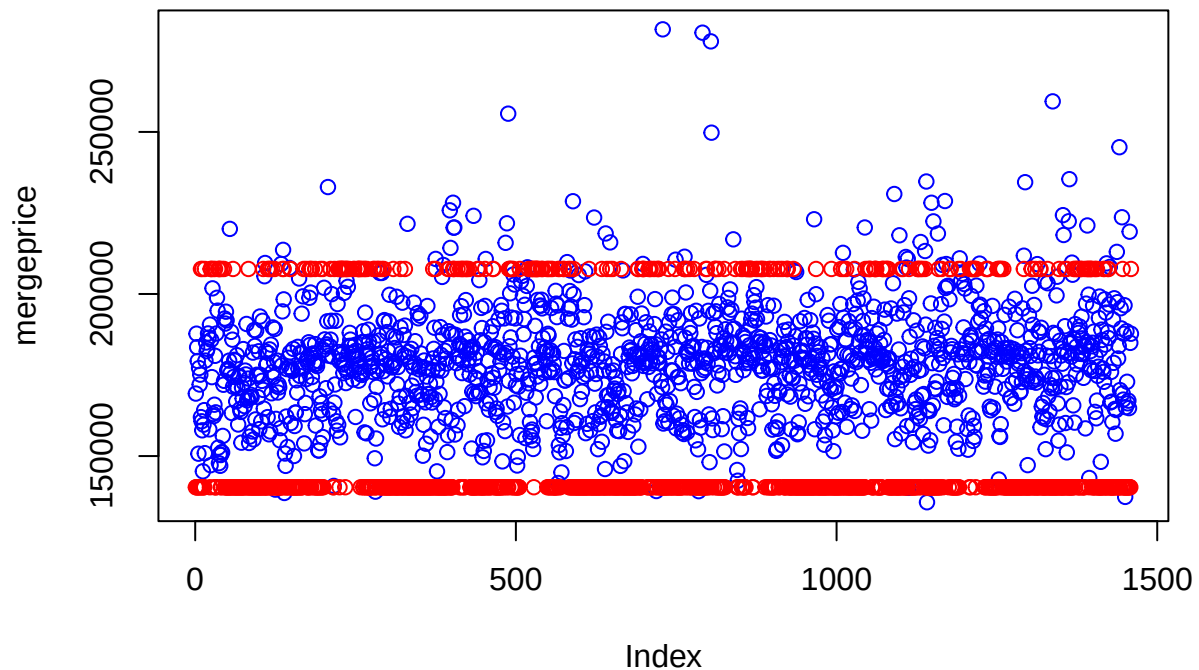
```
## [1] 63229.97
```

```

plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo5, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)

```

Predicciones vs valores originales



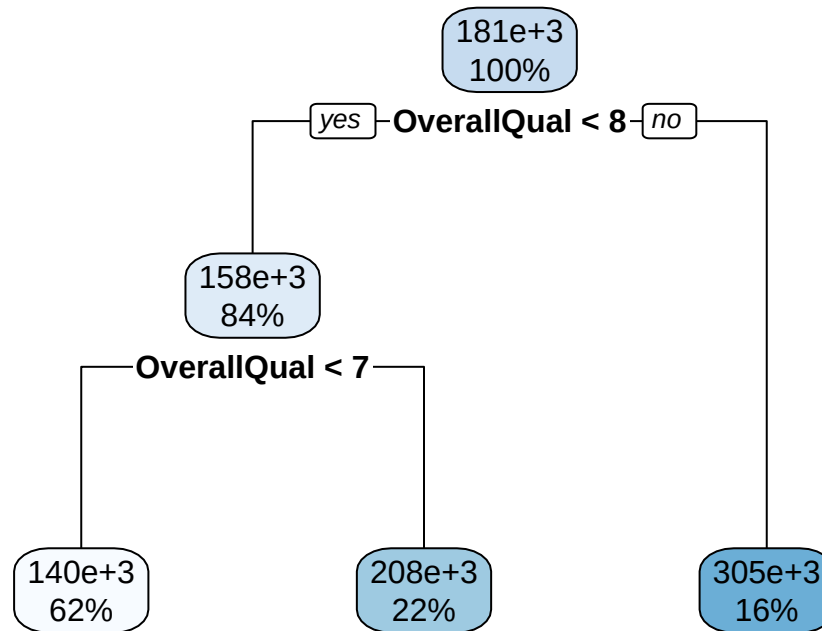
La gráfica mostrada con validación cruzada tiene datos sobreajustados a comparación de la gráfica utilizando Naive Bayes no observamos ese patrón. Por otro lado observamos que el RMSE del modelo de validación cruzada es menor a comparación del Naive Bayes. Por lo tanto, se debe tomar en cuenta que el modelo que funciona más en estos casos es el modelo Naive Bayes debido a que no tiene overfitting de datos lo cual no se ajusta demasiado con el modelo.

9. Tanto para los modelos de regresión como de clasificación, pruebe con varios valores de los hiperparámetros, use el mejor modelo del tuneo, ¿Mejoraron los modelos? Explique

```
controlcv <- trainControl(method = "repeatedcv", number = 8, repeats = 3)
modelo5 <- train(SalePrice ~ ., data = train,
  method = "rpart",
  trControl = controlcv,
  na.action = na.pass
)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
rpart.plot(modelo5$finalModel)
```



```

predModelo5 <- predict(modelo5, newdata = datos)
rmseModelo5test<-RMSE(predModelo5,mergeprice)
mseModelo5test<-mean((predModelo5 - mergeprice)^2)

predModelo5Train <- predict(modelo5, newdata = train)
rmseModelo5train<-RMSE(predModelo5Train,train$SalePrice)
mseModelo5train<-mean((predModelo5Train - train$SalePrice)^2)

```

```
mseModelo5test
```

```
## [1] 3998029436
```

```
## [1] 3998029436
```

```
rmseModelo5test
```

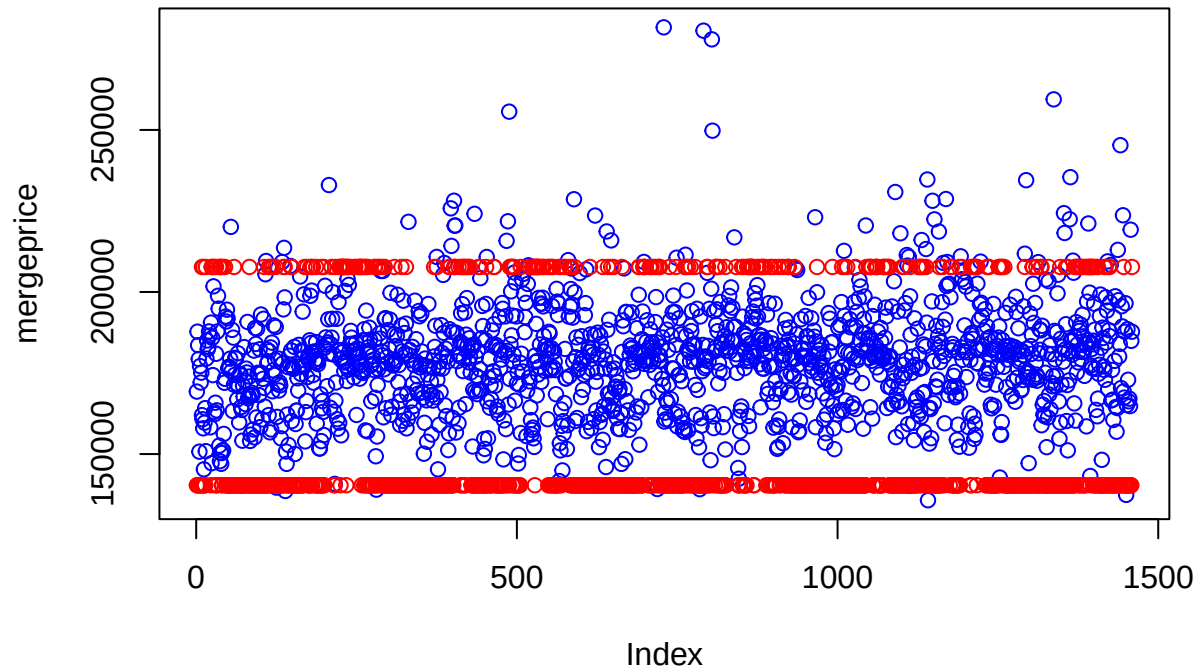
```
## [1] 63229.97
```

```

plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo5, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)

```

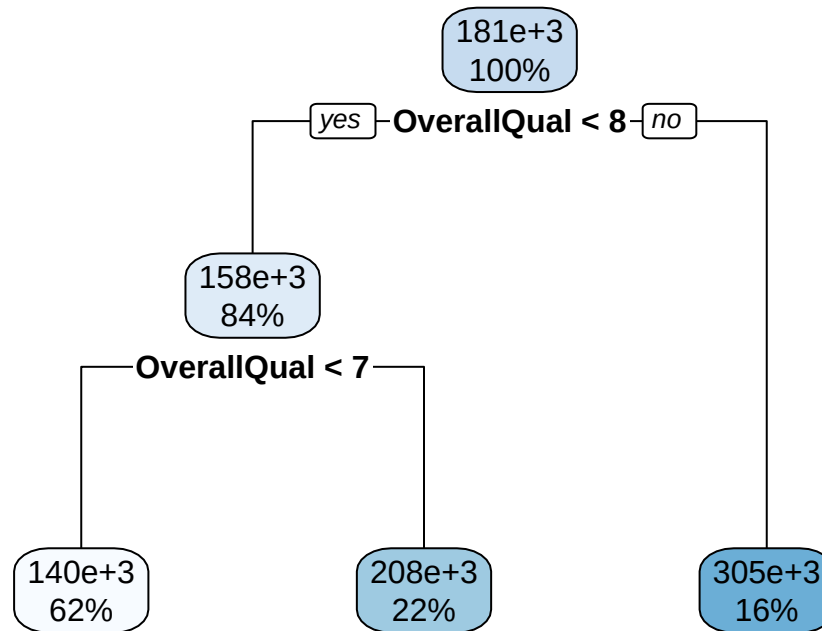
Predicciones vs valores originales



```
controlcv <- trainControl(method = "boot", number = 20)
modelo5 <- train(SalePrice ~ ., data = train,
  method = "rpart",
  trControl = controlcv,
  na.action = na.pass
)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
rpart.plot(modelo5$finalModel)
```



```

predModelo5 <- predict(modelo5, newdata = datos)
rmseModelo5test<-RMSE(predModelo5,mergeprice)
mseModelo5test<-mean((predModelo5 - mergeprice)^2)

predModelo5Train <- predict(modelo5, newdata = train)
rmseModelo5train<-RMSE(predModelo5Train,train$SalePrice)
mseModelo5train<-mean((predModelo5Train - train$SalePrice)^2)

```

```
mseModelo5test
```

```
## [1] 3998029436
```

```
## [1] 3998029436
```

```
rmseModelo5test
```

```
## [1] 63229.97
```

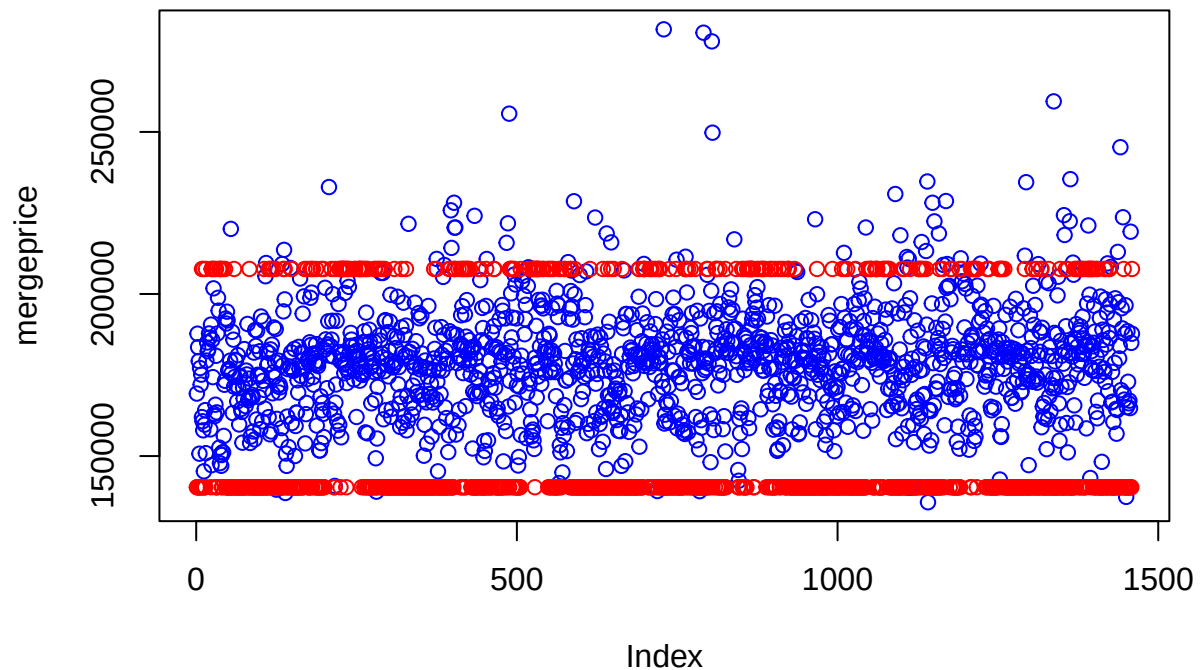
```
## [1] 63229.97
```

```

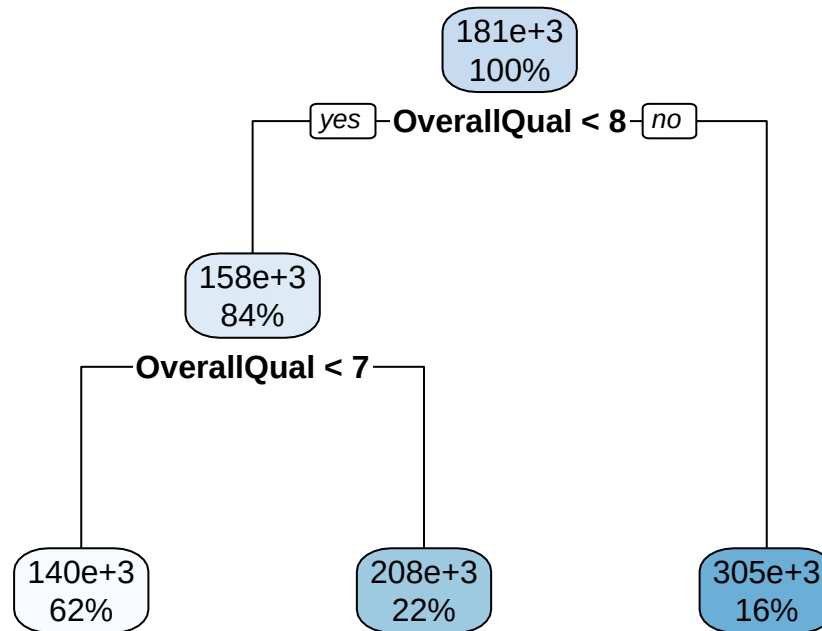
plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo5, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)

```

Predicciones vs valores originales



```
## Warning in adaptiveWorkflow(x = x, y = y, wts = weights, info = trainInfo, :  
## There were missing values in resampled performance measures.  
  
## Warning in data.frame(..., check.names = FALSE): los nombres de fila se  
## encontraron a partir de una variable corta y se descartaron  
## Warning in data.frame(..., check.names = FALSE): los nombres de fila se  
## encontraron a partir de una variable corta y se descartaron  
## Warning in data.frame(..., check.names = FALSE): los nombres de fila se  
## encontraron a partir de una variable corta y se descartaron  
## Warning in data.frame(..., check.names = FALSE): los nombres de fila se  
## encontraron a partir de una variable corta y se descartaron  
  
## Warning in adaptiveWorkflow(x = x, y = y, wts = weights, info = trainInfo, :  
## There were missing values in resampled performance measures.
```



```

predModelo5 <- predict(modelo5, newdata = datos)
rmseModelo5test<-RMSE(predModelo5,mergeprice)
mseModelo5test<-mean((predModelo5 - mergeprice)^2)

predModelo5Train <- predict(modelo5, newdata = train)
rmseModelo5train<-RMSE(predModelo5Train,train$SalePrice)
mseModelo5train<-mean((predModelo5Train - train$SalePrice)^2)

```

```
mseModelo5test
```

```
## [1] 3998029436
```

```
## [1] 3998029436
```

```
rmseModelo5test
```

```
## [1] 63229.97
```

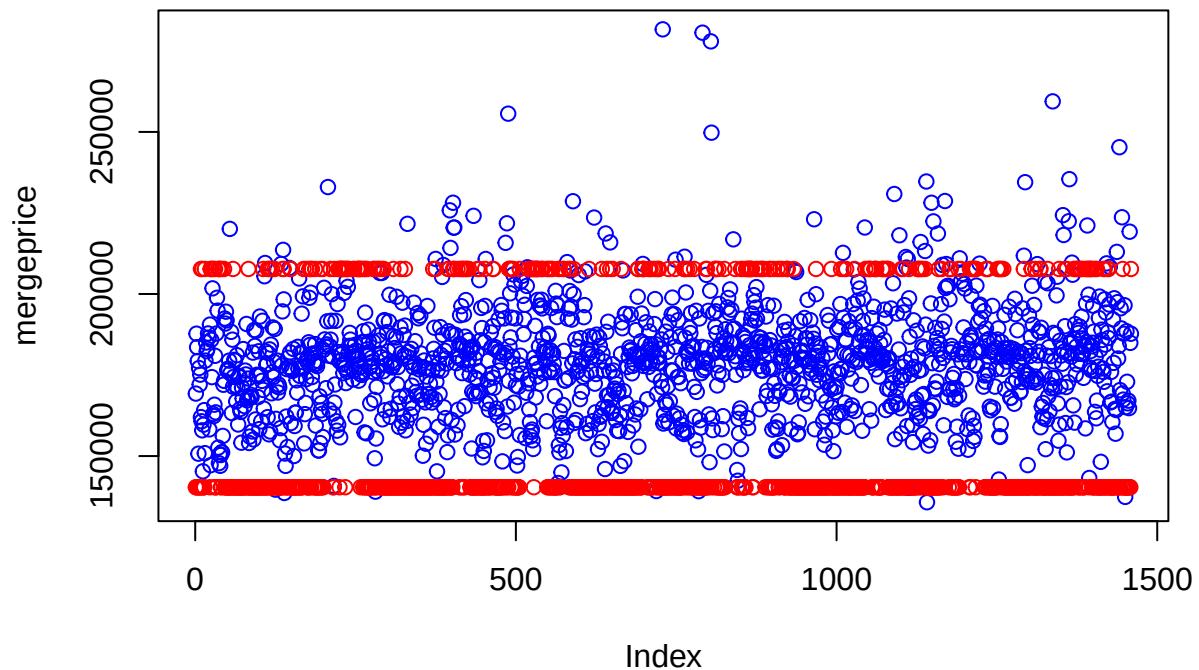
```
## [1] 63229.97
```

```

plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo5, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)

```

Predicciones vs valores originales



Se realizaron diferentes metodos para realizar validación con la finalidad de ver si existen algunas mejoras. Se infiere que no hubo unos cambios encontrados con el RMSE , MSE y además en todas las gráficas se observan el overfitting de los datos.

10. Compare la eficiencia del algoritmo con el resultado obtenido con el árbol de decisión (el de clasificación) y el modelo de random forest que hizo en la hoja pasada. ¿Cuál es mejor para predecir? ¿Cuál se demoró más en procesar?

```
system.time({
  modelo5 <- train(SalePrice ~ .,
    data = train,
    method = "rpart",
    trControl = controlcv,
    na.action = na.pass
  )
})

##      user  system elapsed
##      1.25    0.01    1.30

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
##      user  system elapsed
##      1.80    0.06    1.91

system.time({
  modeloRF1 <- randomForest(SalePrice ~ ., data = train)
})
```

```
##      user  system elapsed
##      4.78    0.04    5.02
##      user  system elapsed
##      4.64    0.05    4.86
```

```
system.time({
  modeloNB <- naiveBayes(SalePrice ~ ., data = train)
})
```

```
##      user  system elapsed
##      0.34    0.01    0.38
##      user  system elapsed
##      0.34    0.05    0.43
```

En terminos de eficiencia, se puede medir con el tiempo y en base a los tres modelos comparados (random forest, Naive Bayes y Random Forest). Observamos que Naive bayes es el más eficiente debido a que el tiempo más corto entre estos modelos. El más costos es el random forest como se puede observar ante los resultados.